Creating and Sharing Genomic Annotation Modules with AI: The Just-DNA Ecosystem

Anonymous submission to EASRP 2026

Abstract

Genome analysis is an iterative workflow—ask, select data, score, annotate, inspect, and revise—and it is increasingly done inside AI coding assistants rather than fixed graphical interfaces. But general-purpose language models are unreliable when used directly for genomics: they fabricate variant identifiers, misassign effect directions, and attach real PMIDs to the wrong paper. The Just-DNA ecosystem redirects these assistants from generating disposable code to producing structured, versioned data: *genomic annotation modules* where every variant is linked to its genotype, evidence, effect size, and a verbatim passage from the source paper. The output is an editable data artifact, not generated analysis code, and a domain expert can review it in a spreadsheet without specialized tools.

This paper presents the plugin ecosystem that makes this possible. The authoring plugin, **just-module-creator**, runs inside the AI coding assistants that bioinformaticians and developers already use daily, so module authoring requires no new infrastructure. The underlying toolchain validates authored files against a live schema, checks identifiers against reference databases, and compiles modules into a portable artifact. A shared registry lets independent authors publish, version, and exchange modules—the package-registry pattern applied to genomic annotation knowledge. Once installed, a module can be applied to a personal genome in under a minute; the annotation consumer, Just-DNA-Lite, is the subject of a companion paper [Anonymous, 2026]. The software is open-source and available at `https://anonymous.4open.science/r/just-dna-registry` (anonymized for review).

1 Introduction

Genomic annotation connects variant calls to the knowledge that gives them meaning: trait associations, clinical classifications, effect sizes, and the evidence behind each claim. This knowledge is scattered across papers, databases, and preprints, and it changes as new studies appear. Turning it into something a computational pipeline can use requires structured, validated records with unambiguous variant identifiers, genome-build coordinates, and traceable citations.

Researchers increasingly use AI coding assistants such as Claude Code and Codex for this kind of work. These tools are powerful, but when asked to analyze variants they typically generate a script: code that may fabricate identifiers [Jin et al., 2024b], attach a real PMID to the wrong paper [Ji et al., 2023, Gao et al., 2023], reverse an allele, or bypass established filters. Each session may produce slightly different code, and the result must be reviewed for correctness, efficiency, and security before it can be trusted. The output is a one-off answer tied to one session, not a reusable resource.

This paper presents the Just-DNA ecosystem, which gives these same assistants a different job. Instead of generating code, the assistant produces structured data: a *module* of editable CSV tables where each variant is linked to its studies, citations, effect sizes, and supporting passages. The input can be anything the assistant can read, from a single sentence describing a trait to a collection of papers or a summary from another model. Every variant is verified against reference databases such as dbSNP; dedicated checking tools validate gene symbols against HGNC, trait identifiers against ontology services, and flag decisions that require domain expertise. Once compiled to Parquet, Just-DNA-Lite joins the module to a personal genome [Anonymous, 2026]. Modules are versioned and shareable: they can be kept locally, rehearsed in a staging registry, or published for others to install and use. The same files can also be written or edited entirely by hand.

This paper makes four contributions:

1. It describes the module contract shared by manual editors, AI-assisted authoring tools, versioned stores, and the genome-analysis consumer.

2. It presents `just-module-creator`, an authoring plugin that exposes the public capabilities of `just-dna-format`, `just-dna-compiler`, `just-dna-enricher`, and `just-dna-registry` as typed Model Context Protocol (MCP) tools [Anthropic, 2024]. The plugin reads the installed schema instead of carrying a copy.

3. It defines how modules move between a local store, a deletable test registry, and the production registry while preserving provenance and decisions that still require review.

4. It defines an evaluation protocol for variant recovery, citation identity, and effect-direction agreement, and reports catalog statistics from the first eight published modules across five independent namespaces.

2 Related work

Genome annotation and catalogs. A variant call records what differs between a sample and a reference genome. Annotation adds information about that call, such as its predicted consequence, known clinical classification, reported trait association, or the conclusion assigned to a particular genotype. VEP, ANNOVAR, and OpenCRAVAT/OakVar perform this kind of lookup by joining a callset to established resources [McLaren et al., 2016, Wang et al., 2010, Pagel et al., 2020]. ClinVar and the GWAS Catalog publish records that can supply annotation content [Landrum et al., 2018, Sollis et al., 2023]. HGMD derives variant–disease associations from the literature through manual review, though the public version updates infrequently and does not include explicit pathogenicity calls [Stenson et al., 2020]. ClinGen provides expert variant curation within the ACMG/AMP framework [Rehm et al., 2015, Richards et al., 2015]. PharmGKB and CPIC supply pharmacogenomic annotations including diplotype-level clinical recommendations [Whirl-Carrillo et al., 2012]. The GA4GH Genomic Knowledge Standards define interoperable representations for variant annotations [Global Alliance for Genomics and Health, 2022]. The problem addressed here begins before the join: selected evidence must be turned into a reviewable, machine-checkable collection that can be distributed and applied consistently.

Table 1 narrows the comparison to systems adjacent to the authoring task: extension-based annotation platforms, structured literature extraction, unconstrained model drafting, and `just-module-creator`.

Biomedical text mining and variant extraction. Conventional named entity recognition systems such as tmVar3 and PubTator3 have improved gene and variant tagging in biomedical text [Wei et al., 2022, 2024]. LitVar 2 links literature paragraphs with variant records from external databases, accelerating knowledge retrieval [Allot et al., 2023]. However, these tools are largely limited to entity-level extraction and do not recover relational information such as variant-specific pathogenicity across disease contexts or the experimental evidence supporting a classification. PubMind applies instruction-tuned LLMs to extract variant–disease–pathogenicity associations directly from over 41 million PubMed abstracts and 5.4 million full-text articles, producing a database of approximately 1.3 million unique variants with contextual annotations [Wang and Wang, 2026]. These systems demonstrate that LLM-based literature mining can recover structured variant knowledge at scale, but their output is a flat database rather than a versioned, schema-validated module that an annotation consumer can install and apply to a genome.

Table 1: Qualitative comparison of systems adjacent to annotation authoring. The rows have different primary tasks; the capability columns compare how their structured outputs are created, checked, and shared. Runtime annotation speed is outside this comparison and is covered in the companion platform paper [Anonymous, 2026].

System	Kind / primary task	*Versioned modules*	*Schema validation*	*Provenance tracking*	*AI-assisted authoring*	*Shared catalog*
OpenCRAVAT / OakVar	annotation platform / user-supplied annotation extensions	✓	✓	–	–	✓
PubMind-DB	text-mined database / LLM extraction of variant–disease–pathogenicity records	–	✓	✓	✓	✓
Raw LLM	general-purpose model / unconstrained drafting	–	–	–	✓	–
just-module-creator	authoring plugin / create, review, and publish installable modules	✓	✓	✓	✓	✓

Criteria. Versioned modules: user-installable or distributable annotation units with explicit versions. *Schema validation*: structured records are checked against a declared machine-readable schema. *Provenance tracking*: record- or module-level source links travel with the output. *AI-assisted authoring*: a language model participates directly in creating or revising the structured output. *Shared catalog*: outputs or extensions are discoverable through a shared service. A dash means the capability is not part of the system's core public workflow, not that no related function can be added around it.

Factuality and tool use. Language models can produce fluent text around a false identifier or citation [Ji et al., 2023]. Benchmarks on genomic question-answering show that models hallucinate gene names, variant identifiers, and functional annotations [Jin et al., 2024b]. Retrieval-augmented approaches and tool use reduce the need to improvise when a model can call a typed operation instead [Schick et al., 2023, Jin et al., 2024a]. The Model Context Protocol provides a standard for connecting models to external tools [Anthropic, 2024]. Biomedical MCP services can search literature, ontologies, and variant databases. just-module-creator uses the same general approach, then carries the result into module files that the rest of the Just-DNA pipeline can inspect and reuse.

Agent orchestration. Many scientific-agent systems assign research and review to several model instances [Hong et al., 2023]. The plugin described here does not require its own orchestration runtime. Claude Code or Codex supplies the agent, while the plugin supplies the tools and the written workflow. This paper evaluates that authoring path, not a particular choice of language model or agent-team topology.

3 Method

3.1 The module artifact

At the artifact boundary, a just-dna annotation module is an editable directory. It always contains `module_spec.yaml` and at least one recognized table. The table records the thing being annotated, such as a genotype, a diplotype, or a measured quantity. Evidence and machine-produced sidecars are stored separately. Compilation converts this human-readable directory into machine-readable Parquet files and a content-addressed `manifest.json` [DNA Seq contributors, 2026]. The compiled artifact is what a consumer installs and joins to a genome.

The authored files are ordinary YAML, CSV, Markdown, and optional image files. A domain expert can open them in a spreadsheet to review and correct an AI draft without special

tools. just-module-creator operates on this same directory rather than introducing a separate format for AI-produced content. The module describes how a genotype or measurement should be interpreted if a consumer encounters it. Just-DNA-Lite, or another compatible consumer, supplies the genome or measurement later.

The author does not need to supply this directory, or any CSV file, at the start of a session. Figure 1 shows how a conversation entry—which can range from a trait name to a collection of papers—moves through routing, scaffolding, and review to a compiled module. The `/create-module` router identifies the appropriate entry point and stage.

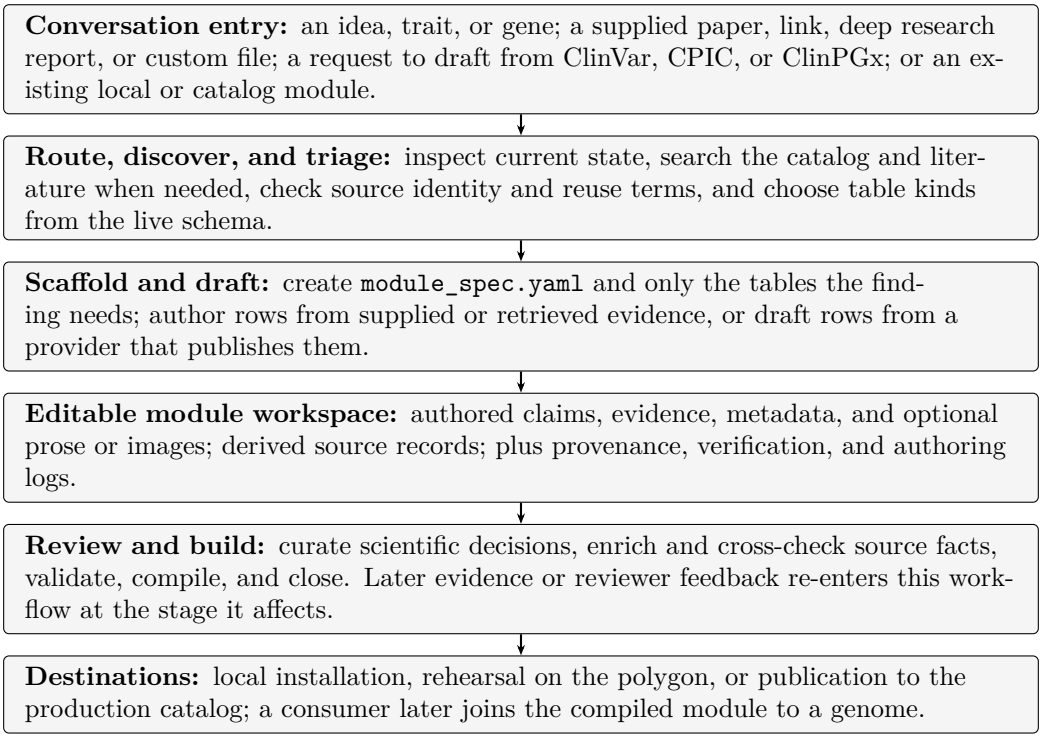

Figure 1: From conversation to a usable module. Module files are intermediate artifacts created by the workflow, not required user inputs. Authored claims remain distinct from derived source records inside the editable workspace.

Different claim types—variant associations, study evidence, pharmacogenomic diplotypes, copy-number ranges—belong to separate tables, each with its own schema. The plugin reads column definitions from the installed schema, so an author always sees the version accepted by the current compiler.

3.2 Architecture and lifecycle

Module authoring and sample analysis remain separate until Just-DNA-Lite joins a compiled module to a local genome. On the authoring path, a person may edit the files directly or use an AI assistant to draft them. Both routes then pass through the same enrichment and compilation steps before the module enters a local or shared store. Figure 2 shows the two paths and the boundary between them.

On the sample path, Just-DNA-Lite converts VCF input to Parquet and joins it with compiled modules using DuckDB, producing annotated reports and enriched data that the user can filter. A whole-genome VCF is annotated in under 40 seconds; the companion paper [Anonymous, 2026] describes the sample path and polygenic risk score computation in detail.

Within `just-module-creator`, the language model participates only in the knowledge path. It can search and read sources, draft module rows, and help review disagreements. VCF filtering,

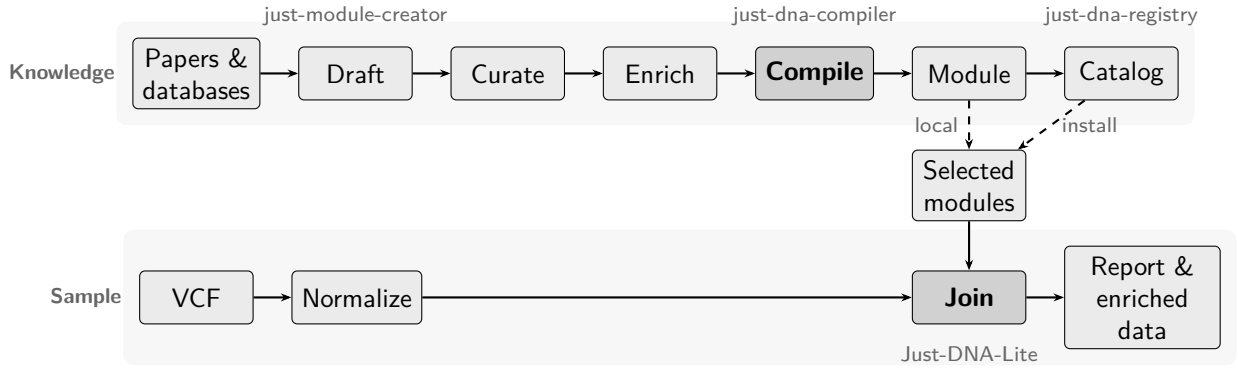

Figure 2: The two paths of the Just-DNA ecosystem. The knowledge path produces a compiled module; the sample path joins selected modules with a local genome via Parquet-to-Parquet joins in DuckDB.

module joins, and polygenic scoring remain ordinary software steps with explicit inputs and repeatable code.

Dependencies point inward: enricher → compiler → format. Table 2 lists each component's responsibility and boundary. `just-module-creator` is the agent-facing authoring entry point examined in this paper, distributed as a plugin for Claude Code and Codex.

Table 2: Responsibilities across the Just-DNA ecosystem.

Component	Responsibility	Does not
`just-module-creator`	research and authoring workflow	process a genome
`just-dna-format`	schema and identity rules	access the network
`just-dna-compiler`	validate, compile, reverse, and sign	access the network
`just-dna-enricher`	resolve, draft, and cross-check	decide scientific meaning
`just-dna-registry`	publish, discover, and download modules	author module rows
Just-DNA-Lite	filter VCFs, join annotations, and produce reports	define module schema or compiler rules

3.3 Determinism, evidence, and responsibility

The compiler establishes structural validity and reproducibility: schema conformance, coordinate resolution, and a deterministic content signature [DNA Seq contributors, 2026]. What it does not establish is whether the authored claims agree with the literature. A false association in valid syntax compiles perfectly. Three properties keep the evidence chain auditable.

First, authored values and their checks are kept independent. The workflow does not fill a value from the source that will later check it, because such a check compares the source with itself. The design test is: *could this check have failed?* If not, it measured nothing (Section 4).

Second, disagreements with reference archives are preserved rather than silently resolved. ClinVar may lag a retraction; when the authored value should remain, `record_override` logs the difference and the reason.

Third, unknown results are distinguished from clean ones. A source timeout produces a null, not a pass. The identifier check writes an attestation to `verification.json` so a reviewer can tell an empty report from one with no findings.

The assistant may locate a verbatim `provenance_quote` from retrieved full text and follow citations into supplementary tables to find per-variant statistics. A `curator` field records who

located each quote—a name or model identifier—so a reviewer can direct scrutiny where it is most needed. Attribution does not transfer responsibility: the human author holds accountability regardless.

Curation follows drafting because a drafted row may still carry decisions the source cannot make—genotype, weight interpretation, conclusion wording—and these are surfaced for domain experts rather than resolved silently.

3.4 Local and shared module stores

A compiled module can reach a consumer through three routes (Figure 2): local installation into a Just-DNA-Lite checkout for testing with a VCF, publication to the staging registry (the polygon, `target=test`) for a deletable rehearsal, or publication to the immutable production catalog. The registry runs its own enrichment and strict compilation on publish, so the stored artifact is the one the server produced, not a locally claimed digest. Inclusion in either catalog distributes a module; it is not scientific review, clinical validation, or endorsement.

3.5 Evaluation dimensions

Three dimensions are relevant for evaluating AI-authored modules against expert-curated ground truth: variant recall (which rsID–genotype pairs were recovered), citation identity (whether each PMID names the intended paper, not merely exists), and weight-sign agreement (whether the effect direction matches). The adjudicated set is not yet large enough for a performance claim; we report these dimensions as a framework for future evaluation.

4 Results

The plugin can be installed from its GitHub repository in a single command (`/install-plugin` in Claude Code, or a repository URL in the Codex plugin marketplace). Once installed, the assistant gains access to 60 typed MCP tools (Appendix B) and 20 skills—written workflow instructions that teach the assistant when to call which tool. In both Claude Code and Codex, skills marked as commands can be invoked directly by typing `/name` in the prompt.

With the plugin active, the assistant calls typed tools instead of generating code (Figure 1): enrichment adds coordinates and derived sidecars, compilation validates against the live schema, and the compiled artifact can be published to the registry and immediately applied to a personal genome. The checks described in Section 3.3 catch citation misattributions, identifier drift, and schema violations in practice—a measurement across 33 upstream `studies.csv` files (44,342 rows) found 3,668 rows where the `provenance_quote` was the article's title, a passage that always matches its own full text and evidences nothing.

4.1 Command menu

Seven of the 20 skills are user-invocable commands, exposing the workflow through intentions rather than internal stages. The remaining thirteen are guides loaded automatically by a router when the assistant reaches the corresponding lifecycle stage.

Command	Purpose
`/create-module`	begin or continue creating a module
`/module-status`	inspect a directory and identify the next decision
`/module-revise`	begin another pass over an existing module
`/find-evidence`	find and read evidence for a row
`/module-publish`	rehearse and then publish
`/module-symptom`	explain a message from the toolchain
`/module-install-local`	install a compiled module without a catalog

4.2 Production catalog

At the time of writing, the production registry holds eight published modules across 19 versions and five independent namespaces (three distinct owners). Table 3 summarizes the published modules.

Table 3: Modules published to the production registry as of August 2026. All modules target GRCh38 and were compiled with strict resolution.

Module	Variants	Studies	Genes	Versions
big_five_personality	330	390	185	4
risk_impulsivity	474	–	325	1
cognitive_intelligence	32	–	31	1
aggression_anger	28	–	22	1
bodybuilding	13	–	13	1
placebo_response (v2)	3	–	3	2
placebo_response_claude	3	–	3	1
lactose_tolerance	2	8	1	2
Total	**885**			**19 versions**

The modules span three namespaces from three owners; three were authored by people outside the development team. Module sizes range from 2 to 474 variants. Four versions of `big_five_personality_snps` show that iterative revision works in practice. Two modules on the same trait (`placebo_response` via Codex, `placebo_response_claude` via Claude Code) were created by the same author to compare the two assistants; both compiled against the same schema and are published side by side.

5 Discussion

Genomic data is becoming personally accessible, and people already use AI assistants—coding tools such as Claude Code and Codex, or plain chat applications such as ChatGPT—to interpret their results [Counts, 2026]. Restricting this is not viable: regulatory frameworks add compliance burdens, yet users find workarounds or proceed informally. The more effective response is to build a community where the work happens in the open, where domain professionals can review what others publish, and where the tools enforce structure. A module is visible, reviewable, and correctable; a one-off chat session is none of these.

The safety concern is real but cuts deeper than AI. Candidate-gene studies have failed to replicate [Border et al., 2019], GWAS effect sizes routinely shrink in independent cohorts [Ioannidis, 2005], direct-to-consumer tests have produced false-positive rates above 40% [Tandy-Connor et al., 2018], and variant reclassifications have led to genetic misdiagnoses [Manrai et al., 2016]. A person from a more deterministic field can easily mistake a statistical association for a causal mechanism. The ecosystem is therefore explicitly research-only, and educating users about these limitations is as important as the tools. A module that faithfully reflects a study which later fails to replicate is not wrong on its own terms, but a user acting on it may be harmed all the same.

The registry turns individual effort into shared infrastructure: authors version, extend, correct, and republish modules—the pattern software engineering solved with package registries. The catalog already holds modules from independent authors, and the plugin's MCP tools are listed on biocontext.ai so other agent platforms can use them independently. Modules contain no individual-level data; Just-DNA-Lite applies them locally, so a person's VCF never leaves their machine and annotation runs raise no GDPR concerns. The knowledge travels through the registry; the genome does not.

Limitations. The module system launched in August 2026 and the catalog is less than a month old. We are developing mechanisms to involve genetic counselors and domain-expert agents in quality evaluation, and to communicate to citizen scientists that many modules reflect early-stage research rather than clinical-grade evidence. Just-DNA-Lite provides an extensive FAQ and science-literacy guide; the module catalog needs comparable guidance. The evaluation contains no creation accuracy estimate; the available fixtures are too small for a performance claim. The authoring workflow targets GRCh38 and does not perform liftover. Runtime benchmarks are in the companion paper [Anonymous, 2026].

6 Conclusion

People are already using AI assistants to interpret genomic data, and the volume of such use will only grow. The Just-DNA ecosystem channels that work into versioned, schema-validated modules that accumulate as shared infrastructure rather than evaporating with each chat session. The registry makes genomic annotation knowledge composable: an author publishes once, and others install, extend, and build on the result. Eight modules from five independent namespaces show that the path from first draft to published, reusable annotation is functional today.

The software is open-source and intended for research use only.

References

Alexis Allot, Kyubum Lee, Qingyu Chen, Ling Luo, and Zhiyong Lu. Tracking genetic variants in the biomedical literature using LitVar 2.0. *Nature Genetics*, 55:901–903, 2023. doi: 10.1038/s41588-023-01414-x.

Anonymous. Just-DNA-Lite: local genome annotation and polygenic risk scoring with deterministic modules, 2026. Companion platform paper (in preparation; citation anonymized for review).

Anthropic. Model context protocol. `https://modelcontextprotocol.io`, 2024. Specification and documentation.

Richard Border, Emma C. Johnson, Luke M. Evans, Andrew Smolen, Noah Berley, Patrick F. Sullivan, and Matthew C. Keller. No support for historical candidate gene or candidate gene-by-interaction hypotheses for major depression across multiple large samples. *American Journal of Psychiatry*, 176(5):376–387, 2019. doi: 10.1176/appi.ajp.2018.18070881.

Aisha Counts. An australian tech entrepreneur used AI to make a cancer vaccine for his dog. *Fortune*, March 2026. URL `https://fortune.com/2026/03/15/australian-tech-entrepreneur-ai-cancer-vaccine-dog-rosie-unsw-mrna/`. Accessed 2026-08-30.

DNA Seq contributors. just-dna-format: schema, compiler, and enricher for just-dna annotation modules. `https://anonymous.4open.science/r/just-dna-compiler`, 2026. Software. Packages `just-dna-format`, `just-dna-compiler`, `just-dna-enricher`.

Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. Enabling large language models to generate text with citations. *arXiv preprint arXiv:2305.14627*, 2023. URL `https://arxiv.org/abs/2305.14627`.

Global Alliance for Genomics and Health. GA4GH genomic knowledge standards. `https://www.ga4gh.org/product/genomic-knowledge-standards/`, 2022. Interoperability standards for variant annotation and interpretation.

Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Zhong Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2023. URL `https://arxiv.org/abs/2308.00352`.

John P. A. Ioannidis. Why most published research findings are false. *PLoS Medicine*, 2(8): e124, 2005. doi: 10.1371/journal.pmed.0020124.

Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023. doi: 10.1145/3571730.

Qiao Jin, Yifan Yang, Qingyu Chen, and Zhiyong Lu. GeneGPT: Augmenting large language models with domain tools for improved access to biomedical information. *Bioinformatics*, 40 (2):btae075, 2024a. doi: 10.1093/bioinformatics/btae075.

Qiao Jin, Yifan Yang, Qingyu Chen, and Zhiyong Lu. GeneTuring tests for the field of computational genomics. *Nature Methods*, 21:768–778, 2024b. doi: 10.1038/s41592-024-02196-4.

Melissa J. Landrum, Jennifer M. Lee, Mark Benson, Garth R. Brown, Chen Chao, Shanmuga Chitipiralla, Baoshan Gu, Jennifer Hart, Douglas Hoffman, Wonhee Jang, Karen Karapetyan, Kenneth Katz, Chunlei Liu, Zenith Maddipatla, Adriana Malheiro, Kurt McDaniel, Michael Ovetsky, George Riley, George Zhou, J. Bradley Holmes, Brandi L. Kattman, and Donna R. Maglott. ClinVar: improving access to variant interpretations and supporting evidence. *Nucleic Acids Research*, 46(D1):D1062–D1067, 2018. doi: 10.1093/nar/gkx1153.

Arjun K. Manrai, Birgit H. Funke, Heidi L. Rehm, Morten S. Olesen, Barry A. Maron, Peter Szolovits, David M. Margulies, Joseph Loscalzo, and Isaac S. Kohane. Genetic misdiagnoses and the potential for health disparities. *New England Journal of Medicine*, 375(7):655–665, 2016. doi: 10.1056/NEJMsa1507092.

William McLaren, Laurent Gil, Sarah E. Hunt, Harpreet Singh Riat, Graham R. S. Ritchie, Anja Thormann, Paul Flicek, and Fiona Cunningham. The Ensembl variant effect predictor. *Genome Biology*, 17:122, 2016. doi: 10.1186/s13059-016-0974-4.

Kymberleigh A. Pagel, Rick Kim, Kyle Moad, Ben Busby, Lily Zheng, Collin Tokheim, Michael Ryan, and Rachel Karchin. Integrated Informatics Analysis of Cancer-Related Variants. *JCO Clinical Cancer Informatics*, 4:310–317, 2020. doi: 10.1200/CCI.19.00132. OpenCRAVAT. OakVar is the maintained successor.

Heidi L. Rehm, Jonathan S. Berg, Lisa D. Brooks, Carlos D. Bustamante, James P. Evans, Melissa J. Landrum, David H. Ledbetter, Donna R. Maglott, Christa Lese Martin, Robert L. Nussbaum, Sharon E. Plon, Erin M. Ramos, Stephen T. Sherry, and Michael S. Watson. ClinGen — the clinical genome resource. *New England Journal of Medicine*, 372(23):2235–2242, 2015. doi: 10.1056/NEJMsr1406261.

Sue Richards, Nazneen Aziz, Sherri Bale, David Bick, Soma Das, Julie Gastier-Foster, Wayne W. Grody, Madhuri Hegde, Elaine Lyon, Elaine Spector, Karl Voelkerding, and Heidi L. Rehm. Standards and guidelines for the interpretation of sequence variants: a joint consensus recommendation of the American College of Medical Genetics and Genomics and the Association for Molecular Pathology. *Genetics in Medicine*, 17(5):405–424, 2015. doi: 10.1038/gim.2015.30.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models

can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Elliot Sollis, Annalisa Mosaku, Ala Abid, Annalisa Buniello, Maria Cerezo, Laurent Gil, Tudor Groza, Osman Güneş, Peggy Hall, James Hayhurst, Arwa Ibrahim, Yue Ji, Sajo John, Elizabeth John, G. Naamati, Michael Nicholson, A. Oyewole, E. Palumbo, Nigel W. Rayner, C. Sacramento, S. Trevanion, M. Zaharia, A. McMahon, L. Harris, H. Parkinson, and F. Cunningham. The NHGRI-EBI GWAS catalog: knowledgebase and deposition resource. *Nucleic Acids Research*, 51(D1):D977–D985, 2023. doi: 10.1093/nar/gkac1010.

Peter D. Stenson, Matthew Mort, Edward V. Ball, Mark Chapman, Katie Evans, Luisa Azevedo, Matthew Hayden, Sally Heber, and David N. Cooper. The Human Gene Mutation Database (HGMD): optimizing its use in a clinical diagnostic or research setting. *Human Genetics*, 139: 1197–1207, 2020. doi: 10.1007/s00439-020-02199-3.

Stephany Tandy-Connor, Jenna Guiltinan, Kate Krempely, Holly LaDuca, Patrick Reineke, Stephanie Gutierrez, Priscilla Gray, and Brigette Tippin Davis. False-positive results released by direct-to-consumer genetic tests highlight the importance of clinical confirmation testing for appropriate patient care. *Genetics in Medicine*, 20(12):1515–1521, 2018. doi: 10.1038/gim201838.

Kai Wang, Mingyao Li, and Hakon Hakonarson. ANNOVAR: functional annotation of genetic variants from high-throughput sequencing data. *Nucleic Acids Research*, 38(16):e164, 2010. doi: 10.1093/nar/gkq603.

Peng Wang and Kai Wang. PubMind: literature-based genetic variant extraction and functional annotation using large language models. *Nature Communications*, 2026. doi: 10.1038/s41467-026-76834-4. Article in Press.

Chih-Hsuan Wei, Alexis Allot, Kevin Riehle, Aleksandar Milosavljevic, and Zhiyong Lu. tmVar 3.0: an improved variant concept recognition and normalization tool. *Bioinformatics*, 38(18): 4449–4451, 2022. doi: 10.1093/bioinformatics/btac537.

Chih-Hsuan Wei, Alexis Allot, Robert Leaman, and Zhiyong Lu. PubTator 3.0: an AI-powered literature resource for unlocking biomedical knowledge. *Nucleic Acids Research*, 52(W1): W540–W546, 2024. doi: 10.1093/nar/gkae235.

Michelle Whirl-Carrillo, Ellen M. McDonagh, J. M. Hebert, Li Gong, Katrin Sangkuhl, Caroline F. Thorn, Russ B. Altman, and Teri E. Klein. Pharmacogenomics knowledge for personalized medicine. *Clinical Pharmacology & Therapeutics*, 92(4):414–417, 2012. doi: 10.1038/clpt.2012.96.

7 Code and data availability

- **just-dna-format, just-dna-compiler, and just-dna-enricher**: `https://anonymous.4open.science/r/just-dna-compiler` (schema, reference compiler, and enricher).
- **just-module-creator**: `https://anonymous.4open.science/r/just-dna-registry` (MCP server, Claude Code and Codex plugin manifests, skills; MIT).
- **Just-DNA-Lite**: `https://anonymous.4open.science/r/just-dna-lite` (local VCF processing, annotation, and reporting; described in the companion paper [Anonymous, 2026]).
- The production catalog and staging polygon are live at URLs provided in the anonymized repositories.

Appendix

A Research use only

Annotation modules summarize published association findings. They are not clinical-grade evidence. The authoring plugin does not make individual-level predictions and does not open a genome. Just-DNA-Lite and `just-prs` process sample data for research and educational use; their reports and scores are not diagnoses or medical advice. Language that implies causation, such as "causes" or "guarantees," is outside the scope of a module written with this plugin.

B Plugin surface: tools and skills

A Claude Code or Codex plugin is a bundle of two kinds of asset. **MCP tools** are typed operations the assistant can call—each has a name, typed inputs, and a structured return value. **Skills** are written instructions loaded into the assistant's context on demand; they teach the workflow that the tools serve, including when to call which tool and what to do with the result. Together, 60 tools and 20 skills make up the `just-module-creator` plugin surface.

Tools (60)

The tools are organized into a core set (always listed) and nine optional groups that a session can reveal incrementally via `toolbox`. Table 4 lists the 18 core and meta tools; Tables 5–13 list the nine groups. Registry writes require a token obtained through `authenticate`. Literature and enrichment requests share a single pacing gate, including the NCBI budget.

Table 4: Core tools (always listed) and the meta-tool.

Tool	Purpose
`list_tables`	List authorable table kinds and what each row represents
`describe_table`	Describe one table kind: every column, type, vocabulary, and pick-list
`table_requirements`	The three shapes of requiredness for a table kind
`describe_machine_table`	Describe a machine-written sidecar: columns, types, vocabulary
`get_template`	Get a CSV template (header only, or header plus stub rows)
`scaffold_module`	Create `module_spec.yaml` plus stub CSVs; never overwrites
`lint_rows`	Lint CSV text against a table kind (writes nothing)
`validate_module`	Pre-flight a spec directory (writes nothing)
`compile_module`	Compile a spec directory into Parquet plus `manifest.json`
`draft_from_clinvar`	Draft `variants.csv` and `studies.csv` from ClinVar
`enrich_module`	Resolve rsIDs to coordinates and mint VRS identifiers
`literature_search`	Find papers behind a row; confirm a PMID names the intended paper
`lookup_citation`	Check a PMID or DOI and read back which paper it names
`lookup_variant`	Look up one variant: loci, alleles, ClinVar calls, rsID currency
`record_override`	Log that an authored value deliberately outranks a source
`registry_register`	Create a registry account and mint its API key
`authenticate`	Provide a registry token to unlock write tools for this session
`toolbox`	List optional tool groups and reveal them to the session

Table 5: Evidence group: read a paper, its citations, and its supplementary tables.

Tool	Purpose
`lookup_open_access`	Where a paper may legally be read, and on what terms
`fetch_fulltext`	Retrieve a paper's text so the assistant can read and quote it
`paper_citations`	Papers that cite this one, or the papers it cites
`list_supplementary`	Inventory the supplementary files attached to a paper
`fetch_supplementary`	Download one supplementary file to the cache
`describe_supplementary`	Sheets in a downloaded workbook and their column names

Table 6: Identifiers group: check gene symbols and ontology CURIEs.

Tool	Purpose
`check_identifiers`	Check every gene symbol (HGNC) and trait CURIE (OLS4) in a spec
`lookup_identifier`	Check one gene symbol or trait CURIE

Table 7: Pharmacogenomics (PGx) group: draft from curated PGx sources.

Tool	Purpose
`draft_from_cpic`	Draft haplotypes, allele function, and diplotypes from CPIC
`draft_from_clinpgx`	Draft `pharm_variants.csv` from a ClinPGx snapshot

Skills (20)

Skills are markdown documents loaded into the assistant's context when needed. Seven are **commands**—user-invocable via `/name` in the assistant's prompt—and thirteen are **guides** loaded automatically by a router when the assistant reaches the corresponding stage.

Table 8: Passes group: fill or re-derive machine-written sidecars.

Tool	Purpose
`enrich_facts`	Fill frequency, constraint, and dosage sidecars
`enrich_literature_pass`	Resolve every citation in `studies.csv` into `literature.csv`
`enrich_gwas_effects`	Record the GWAS Catalog's published effect sizes for the module's rsIDs
`refresh_sidecar`	Re-derive one sidecar against its source, keeping curated rows

Table 9: Review group: read a module back and surface decisions.

Tool	Purpose
`audit_module`	Ask what a curation pass would ask (offline, reads only)
`review_queue`	Rank the rows where somebody overruled a source
`review_logs`	Show what is in the module's logs (publishing them is permanent)
`study_facts`	Per-study facts the GWAS pass wrote into `gwas_effects.csv`

Table 10: Integrity group: compare, sign, verify, and reverse modules.

Tool	Purpose
`compare_modules`	What moved between two spec directories (offline)
`compare_to_published`	Is local data ahead of the catalog, and how?
`module_signature`	Content signature of raw authored data (no compile, no network)
`verify_artifact`	Re-hash every file in a compiled artifact and recompute the digest
`reverse_module`	Turn a compiled artifact back into an authored spec directory

Table 11: Catalog group: read the registry.

Tool	Purpose
`registry_search`	Search published modules (read-only, no token)
`registry_get_module`	Fetch one module's full record: card, readme, versions, manifest
`registry_health`	Is this registry instance up, and which instance is it?
`registry_is_published`	Has this authored data been published under any name?
`registry_namespace_available`	Check whether a namespace is legal and unclaimed
`registry_download`	Download and integrity-verify a published module version

Table 12: Publish group: write to the registry (token required).

Tool	Purpose
`registry_whoami`	Confirm the token is accepted and report the account
`registry_validate`	Validate a spec on the registry without publishing
`registry_check`	Full publish dry run: validation plus network checks
`registry_publish`	Publish a spec directory as a module version
`registry_amend_readme`	Replace a published version's readme (no version bump)
`registry_claim_namespace`	Claim a publishing namespace (irreversible on production)
`registry_yank`	Stop recommending a version (stays fetchable for pinned users)
`registry_unyank`	Restore a yanked version to listings
`registry_delete_version`	Hard-delete one rehearsed version from the polygon
`registry_delete_module`	Hard-delete all rehearsed versions from the polygon

Table 13: Closing group: finish and describe.

Tool	Purpose
`close_module`	Declare authoring finished, bound to its authored bytes
`describe_spec_file`	Describe `module_spec.yaml`: every key and block it carries
`authoring_reference`	The complete generated description of the authoring DSL as JSON

Table 14: User-invocable commands (7). Each is typed as `/name` in the assistant prompt.

Command	Purpose
`/create-module`	Router: begin or continue creating a module from any starting point
`/module-status`	Inspect a directory and identify the next decision
`/module-revise`	Begin another pass over an existing module
`/find-evidence`	Find and read evidence for a row
`/module-publish`	Rehearse on the polygon, then publish to the catalog
`/module-symptom`	Explain a message from the toolchain
`/module-install-local`	Install a compiled module into Just-DNA-Lite locally

Table 15: Guides (13). Each is loaded by a router or a preceding skill rather than invoked by the user directly.

Guide	Purpose
`module-101`	High-level map: what a module is, what the plugin can and cannot do
`module-start`	Stage 0–1: triage, licence, the spec
`module-draft`	Stage 2: draft rows from evidence or a provider
`module-curate`	Stage 3: curate scientific decisions
`module-enrich`	Stage 4: enrich with coordinates, identifiers, and sidecars
`module-check`	Stage 5: cross-check against reference databases
`module-compile`	Stage 6: validate and compile to Parquet
`module-close`	Stage 6b: declare authoring finished and bind to authored bytes
`module-refresh`	Re-run anything that already ran
`module-diff`	What moved, and the reading that means a source changed its answer
`module-tables`	Which table kind, where every file sits, the three on-disk shapes
`module-weights`	The weight column everyone fills and nobody declares
`module-consumer`	The annotation consumer's side of the seam